

EE417 - Assignment №2

Adil Ismagambetov
School of Engineering and Digital Sciences
Nazarbayev University
Astana, Kazakhstan
adil.ismagambetov@nu.edu.kz

I. INTRODUCTION

This assignment required to choose 2 objects and train an object detection model that classifies the object in the frame and draw corresponding bounding box. As my solution, I choose a camera (0) and headphones (1). This report will explain the whole pipeline from data collection and augmentation to models training and their performance comparison.

II. DATA COLLECTION

Overall 58 pictures of these two objects were taken with different backgrounds and from different angles. Proportion of classes is 50% for camera and 50% for headphones. These pictures were taken with the phone camera with an aspect ratio of 1X1. This was done to avoid ruining the aspect ratio of images during rescaling them to 224X224. Therefore, no class imbalance was encountered. In addition, all the pictures were taken under good light conditions. Once the pictures were taken, I resized them to 224X224 using Open-CV2 and labeled them using LabelBox platform.



Fig. 1. Part of Dataset before Data Augmentation.

III. DATA AUGMENTATION

After labeling the data, I saved it in YOLO format because this makes it easier to modify the labels when applying image augmentations like flipping and rotation. For other data augmentation techniques labels do not change. Therefore, I first went through each image and flipped it horizontally, vertically, and rotated it 45 degrees to the right and 45 degrees

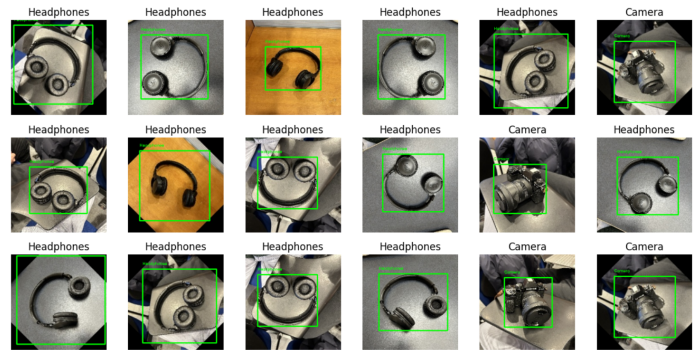


Fig. 2. Part of Dataset after flipping and rotation.

to the left. After this procedure, the dataset size increased from 58 to 290 images.

Labels for these images were adjusted accordingly. For horizontal flip, only X value from label was updated, which is $1 - X_{old}$. The same is true for vertical flip, but with a Y value. For rotations special function "rotate_yolo_label" was written in the data processing file which adjusts the label according to rotation angle.

After this step I applied sharpening, blurring, brightening, darkening, hue augmentation and saturation augmentation to each of 290 images. This resulted in total of 2030 images which were randomly divided as 1218 training images, 406 validation images and 406 testing images.

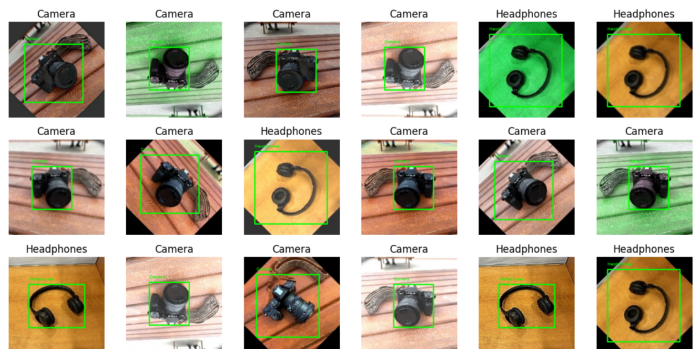


Fig. 3. Part of Dataset after flipping and rotation.

IV. MODELS

I started with a two-stage detector from scratch. As a backbone I choose ResNet18 CNN for Faster-RCNN. I used Adam optimizer with learning rate of 1E-4 and trained 10 epochs. After each epoch I recorded training and validation loss to track the moment of overfitting. According to the graph below, the model started to overfit after epoch 8. In my training loop, I was saving the best model based on min validation loss, therefore, next metrics will be coming not from the last trained model, but from the best trained one.

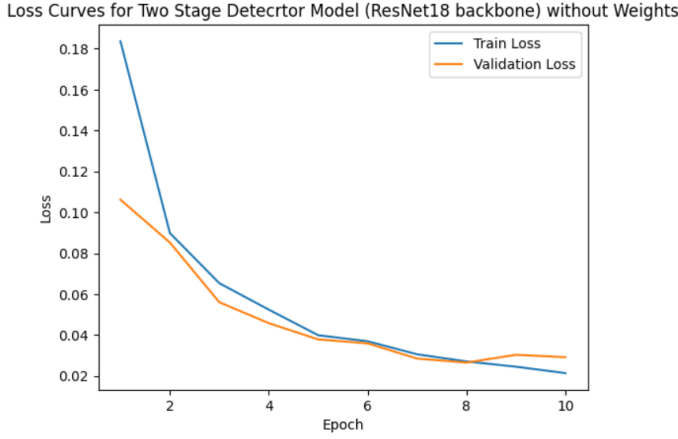


Fig. 4. Train and Validation Loss for Unweighted Two-stage Detector model.

The mean average precision for this model turned out to be 0.42. Precision - 0.365, recall - 0.49, F1-score - 0.42. I couldn't interpret correctly confusion matrix for 2 classes unfortunately.

After that, I started training the same model, but with pretrained weights. I used the same optimizer with learning rate of 1E-4. After 10 epochs my validation and training loss were lower compared to previous model, however, they were quite the same which meant that model is underfit. I decided to train for additional 5 epochs until the point of overfitting. At epoch 15 it was still giving signs of underfit, but since I got no time I decided to implement that model.

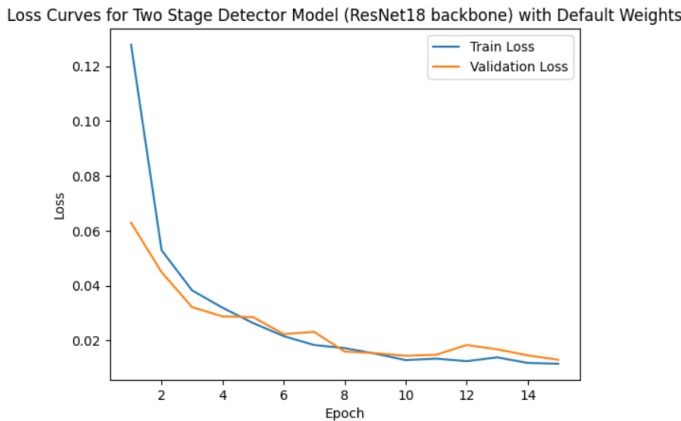


Fig. 5. Train and Validation Loss for Weighted Two-stage Detector model

We can clearly observe that the loss for weighted model was lower. It's metrics are also better, with mAp - 0.46, Precision - 0.44, Recall - 0.49 and F1-Score - 0.46. In both models batch size of 4 was used. I haven't experimented with different learning rate and batch size because of the decent-for-ResNet-18 results.

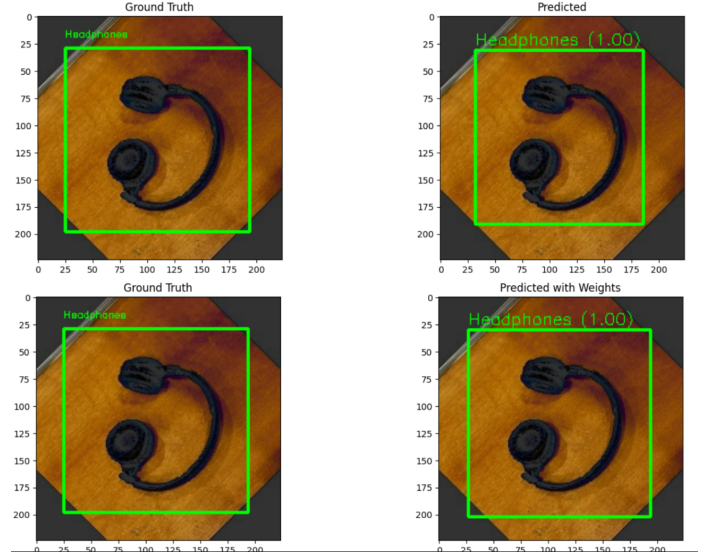


Fig. 6. Bounding Box Prediction for Faster RCNN without weight (top) and with weights (bottom)

The next step was to implement one-stage detector. I choose YOLOv5 for this purpose. If the previous models were implemented on my laptop, I implemented YOLOv5s models using Google Collaboratory because of the hard setup. This architecture is much faster, and therefore I decided to train it with 100 epochs. It took 5 minutes compared to 63 minutes for Faster RCNN. However, it is less accurate with mAP of 0.296. Other metrics will be displayed in the table at the end. In addition, the same model with pretrained weights and 100 epochs resulted in mAP 0.33. It is possible that both models were overfit, or that I calculated the metrics incorrectly. In both cases I have used Adam optimizer and batch size of 64. I haven't tried other combinations.

TABLE I
MODEL PERFORMANCE COMPARISON

Model	mAP	Precision	Recall	F1-score
Faster RCNN	0.42	0.37	0.49	0.42
Faster RCNN (Pretrained)	0.46	0.44	0.49	0.46
YOLOv5	0.30	0.49	0.49	0.49
YOLOv5 (Pretrained)	0.33	0.49	0.49	0.49

Note: All metrics calculated at IoU threshold 0.5.

V. CONCLUSION

This progress report demonstrated the pipeline of the take-home assignment. In my case, Pretrained Faster RCNN resulted in the highest performance with mAP of 0.46. Used



Fig. 7. Pretrained YOLOv5s Predictions on Test (unseen) images

Dataset can be accessed by this link. One potential problem of my work is lack of usage of YOLO models. I haven't figured out fully how to extract the metrics from the model and see whether the model is underfit or overfit. Another issue that was encountered is the confusion matrix. While I understand how to work with confusion matrix in binary cases, with 2 classes I couldn't imagine it.